Lab Deep Dives 1 — Git, Change Requests, Concept Location, Impact Analysis, CI

> Deep, exam-oriented treatments of the six Lecture 1–3 labs: the IntroLab and Git Lab (course bootstrap and version control), the Change Request Lab and Concept Location Lab (the first two phases of a software change), and the Impact Analysis Lab and CI Lab (estimating change scope and automating the build). Each lab gets the same seven-part treatment: the assignment as the handout states it, a concrete walkthrough, report-ready justifications in the decision → rejected alternative → benefit pattern, a fill-in reflection paragraph, likely exam questions with answers, pitfalls, and theory links. Citations: lab handouts as (IntroLab p.1), (Git Lab p.X), (ChangeReqLab p.1), (CLLab p.1), (AnalysisLab p.1–2), (CILab p.1); decks as (ConceptLocation p.X), (ImpactAnalysis p.X), (ContinuousIntegration p.X), (JHotDraw p.X), (ChangeInitiation p.X), (SoftwareChange p.X), (Introduction p.X); exam facts as (What-To-Expect p.N); readings as [Raj13]. Practical steps that go past the handouts and decks are labelled "(beyond slides — practical knowledge)". First-person [placeholders] appear only in the copy-paste reflection sections.

How the labs map to the exam report

The exam is a reflective report written on the day from a handed-out template, and the template's questions are explicitly "related to our lab work and code" (What-To-Expect p.1). The lecturer's own summary is blunt: "main report is how you done the work" (What-To-Expect p.6). That makes the labs — not the lecture theory — the main graded artifact. The theory questions exist "to have more examination," but the bulk of the marks ride on narrating and justifying what you actually did in these lab exercises (What-To-Expect p.6).

Two artifacts are graded together. The **report** must explain how you implemented things, why you made certain technical decisions, and connect the practical work to maintenance and Clean Code concepts (What-To-Expect p.1). The **GitHub repository** is also evaluated in its own right — code changes, refactoring work, testing, pipeline setup, and overall maintenance work (What-To-Expect p.5). Every lab below therefore ends in something checkable: a fork, a branch history, an Issue, a Projects card, a portfolio table, or a workflow file. The repository is the evidence; the report is the argument.

The answer pattern the examiner wants is fixed: not only *what* you did, but especially *why* you did it and *how* you did it (What-To-Expect p.1). The lecturer's worked example — "I used a switch statement instead of multiple if-statements because it improved readability and provided clearer structure" — is a three-part shape: decision, rejected alternative, concrete benefit (What-To-Expect p.4). Every "Why & how" section below pre-builds sentences in exactly that shape for the lab's actual decisions.

Finally, preparation is legal and expected: you may bring prewritten templates, drafts, notes, and diagrams, and you may copy-paste parts of prepared material into the report — only generative AI is banned (What-To-Expect p.2, p.6). The "Copy-paste reflection" section under each lab is written to be pasted and personalized during the exam. One known template question is already about a lab in this file: "how did you create a pipeline" was the final question in an example report the lecturer showed (What-To-Expect p.6) — the CI Lab section answers it.

How the six labs line up with likely report sections:

Lab	Change-process phase it exercises	Likely report topic it feeds
IntroLab	Environment baseline (precondition for everything)	"Describe your project setup / repository"
Git Lab	Conclusion-phase mechanics (commit, baseline) + collaboration	"How did you use version control?"
ChangeReqLab	Initiation — the change request	"What change did you implement and why?"
CLLab	Concept Location	"How did you find the code to change?"
AnalysisLab	Impact Analysis	"How did you assess the impact of your change?"
CILab	Verification automation / build	"How did you create your pipeline?" (What-To-Expect p.4, p.6)

Intro Lab (Lecture 1) — forking JHotDraw, Maven build, first run

The assignment — what the handout asks

The IntroLab is a one-page handout whose stated purpose is to "check-out our CASE study source code for the course" (IntroLab p.1). Its two objectives are getting started with Maven and getting started with the GitHub workflow (IntroLab p.1). The classwork is a fixed sequence of five tasks (IntroLab p.1):

1. Each **team** creates a GitHub **fork** of the project repository, linked in the handout as [JHotDraw].
2. In future lab exercises, each team member follows **GitHub flow** to collaborate — that is, each member works on **their own feature branch**.
3. Install the **Maven build system 3.8.x based on JDK 11**.
4. Go to the project root folder and run `mvn clean install -DskipTests`.
5. Execute `mvn exec:java "-Dexec.mainClass=org.jhotdraw.samples.svg.Main"` at the `jhotdraw-samples-misc` folder level via the terminal — after which "the JHotDraw software should now be showing a GUI" (IntroLab p.1).

There is no separate portfolio artifact for this lab; its deliverable is a working environment — a team fork that builds and runs. Everything later in the course (user stories, debugger sessions, impact walks, pipelines, refactorings) presupposes exactly this baseline.

Step-by-step walkthrough — how to do it

1. Fork the repository. Open the course's JHotDraw repository link on GitHub while signed in, press *Fork*, and create the fork under the team's account or organization so every member has push access (IntroLab p.1). The fork — not the upstream — is from now on "the project": it is the central remote the team shares, the place Issues and Projects boards live, and the repository the examiner will grade (What-To-Expect p.5). (beyond slides — practical knowledge) Add each team member as a collaborator under *Settings → Collaborators* so everyone can push branches.

2. Clone your fork. On your machine: `git clone https://github.com/<team>/JHotDraw.git`. Cloning creates a complete local copy of the whole repository — all files and full history — and links it to the remote under the default name **origin** (Git Lab p.15). From this point the Git four-area model applies: workspace, index, local repository, remote (Git Lab p.11).

3. Verify the toolchain. The handout pins Maven 3.8.x on JDK 11 (IntroLab p.1). (beyond slides — practical knowledge) Check `java -version` reports a JDK 11 build and `mvn -version` reports both Maven 3.8.x and

the JDK it picked up — Maven uses `JAVA_HOME`, so if `mvn -version` shows JDK 17 while `java -version` shows 11, set `JAVA_HOME` to the JDK 11 install before building. Version drift here is the single most common cause of first-build failures, because a newer JDK can reject the project's compiler settings even though nothing is wrong with the code.

4. Build: `mvn clean install -DskipTests` from the project root. Unpacking the command (beyond slides — practical knowledge): `clean` deletes each module's `target/` directory so the build starts from source only; `install` runs the full lifecycle (compile, package) and then copies each built artifact into your local Maven repository (`~/.m2/repository`); `-DskipTests` sets the property that makes Surefire skip test execution. JHotDraw is a **multi-module** Maven project — the root POM is a reactor that builds the modules in dependency order — which is exactly why the handout says `install` and not just `compile`: later modules (like the samples) resolve earlier modules (like the core) from the local repository, so they must be installed there first. Expect the first build to be slow while Maven downloads dependencies; subsequent builds are much faster because the artifacts are cached in `~/.m2`.

5. Run the SVG sample. Change into the `jhotdraw-samples-misc` folder and run `mvn exec:java "-Dexec.mainClass=org.jhotdraw.samples.svg.Main"` (IntroLab p.1). The `exec:java` goal runs a main class *inside the module's own classpath*, which is why the command must be issued at the `jhotdraw-samples-misc` level: that module is where `org.jhotdraw.samples.svg.Main` lives and where its dependencies are declared (beyond slides — practical knowledge). The quoting matters on some shells — the whole `-Dexec.mainClass=…` token is passed as one argument. If everything worked, the JHotDraw SVG sample editor opens: the drawing application whose annotated screenshot the JHotDraw deck uses to teach the framework vocabulary — **Tools** on the toolbar, **Figures** on the canvas, the **Drawing** as the canvas itself (JHotDraw p.4, p.6).

6. Commit the baseline. Per the Git Lab's TODOs, commit your initial project code with `add`, `commit`, `pull`, `push` (Git Lab p.19). (beyond slides — practical knowledge) Before the first commit, make sure build output stays out of history: confirm `.gitignore` covers `target/` — committing generated artifacts is precisely the "don't do it" the Git deck warns about with its crossed-out `.EXE` and 2 GB file (Git Lab p.6).

7. Smoke-test the GUI. Click through the features named on the demo drawing — connections, groups, text, annotation, images, URL attachments (JHotDraw p.4, p.6). This five-minute tour pays off twice: it confirms the build really works, and it is your first look at the menu of **existing features** the ChangeReqLab will ask you to write a user story about (ChangeReqLab p.1).

Why & how — report-ready justifications

- We **forked** the repository instead of cloning the upstream directly because a fork gives the team its own writable remote with Issues, Projects, and pull requests, whereas working against the upstream would have left us without push rights and without a gradable repository of our own (IntroLab p.1; What-To-Expect p.5).
- We built with `mvn clean install` rather than `mvn compile` because JHotDraw is a multi-module project: `install` places each module's artifact in the local repository so dependent modules resolve against it, while plain compilation would have left inter-module dependencies unresolvable (beyond slides — practical knowledge).
- We used `-DskipTests` for the bootstrap build instead of running the full test suite because the goal of day one was a verified compile-and-run baseline, not verification of behavior; test execution was deferred to the CI pipeline, where it runs automatically on every pull request (IntroLab p.1; CILab p.1).
- We pinned **JDK 11 with Maven 3.8.x** rather than letting each member use whatever was installed because identical toolchains make builds reproducible across machines and eliminate "works on my machine" failures before they start (IntroLab p.1; ContinuousIntegration p.6).

- We adopted **GitHub flow with one feature branch per member** from the very first lab, rather than committing to main, because isolating concurrent work keeps main releasable and makes each person's contribution separately reviewable — which matters when the repository itself is graded (IntroLab p.1; What-To-Expect p.5).
- We ran the SVG sample through `mvn exec:java` from `jhotdraw-samples-misc` rather than configuring an IDE run configuration first, because the terminal command works identically on every machine and proves the *Maven* build is self-sufficient — the IDE setup came later as a convenience, not a dependency (IntroLab p.1; beyond slides — practical knowledge).

Copy-paste reflection for the report

In the introduction lab I established the technical baseline that every later change relied on. Our team forked the JHotDraw repository on GitHub so we had a writable remote of our own, and I cloned the fork locally with `git clone`, which gave me the complete history and linked the team fork as origin (Git Lab p.15). I installed Maven 3.8.x on JDK 11 as the handout required and verified the versions with `mvn -version` before building, because a mismatched JDK was the most likely source of spurious build failures (IntroLab p.1). I built with `mvn clean install -DskipTests` from the project root — install rather than compile because JHotDraw is multi-module and dependent modules resolve installed artifacts from the local repository — and deferred test execution to the CI pipeline we built later. I then ran the SVG editor with `mvn exec:java "-Dexec.mainClass=org.jhotdraw.samples.svg.Main"` from jhotdraw-samples-misc and confirmed the GUI appeared. [One problem I hit was — e.g. JAVA_HOME pointing at the wrong JDK / a failed dependency download —] which I resolved by [fix]. Exploring the running editor — its tools, figures, and the demo drawing's features — gave me the concrete feature list I later drew on for my change request, [feature], and meant that when I started concept location I already knew the application's behavior from the user's side.

Likely exam questions about this lab — with answers

Q: Why did you fork JHotDraw instead of just cloning the original repository? A fork creates the team's own copy on GitHub with full push rights, its own Issues tracker, Projects board, and pull-request workflow (IntroLab p.1). Cloning the upstream alone would give a local copy but no shared writable remote, so the team could not collaborate through pushes and pull requests. The fork is also the artifact the examiner grades — the repository's code changes, branches, and pipeline are all evaluated (What-To-Expect p.5).

Q: What does `mvn clean install -DskipTests` actually do? `clean` deletes the generated `target/` directories so the build starts from pure source; `install` compiles, packages, and copies each module's artifact into the local Maven repository in `~/.m2`; `-DskipTests` tells the test plugin not to execute tests during this build (beyond slides — practical knowledge). Installing matters because JHotDraw is multi-module: later modules resolve earlier modules from the local repository. Skipping tests is a bootstrap-speed decision — verification is the pipeline's job, not the first build's (CILab p.1).

Q: Why must the run command be executed at the `jhotdraw-samples-misc` folder level? The `exec:java` goal runs a main class on the classpath of the module where it is invoked. `org.jhotdraw.samples.svg.Main` belongs to the samples module, so only there are the class and its dependencies on the classpath (IntroLab p.1; beyond slides — practical knowledge). Running it from the root would either fail to find the class or assemble the wrong classpath.

Q: What is "origin" and where did it come from? `origin` is the default name Git gives the remote repository a clone was created from (Git Lab p.15). In our setup it points at the team's JHotDraw fork on GitHub, so `git push` publishes my local commits to the team and `git pull` brings teammates' published commits into my workspace. The name is a convention, not magic — additional remotes can be linked later.

Q: How does this lab connect to the software-change process the course teaches? It creates the baseline the whole change mini-cycle operates on: you cannot locate concepts, analyze impact, or verify changes in a system you cannot build and run. It also fixes the course's technical environment — JHotDraw as the case study, Maven as the build, Git/GitHub as the change record (Git Lab p.18–19; IntroLab p.1) — so that every later phase, from Initiation to Conclusion, leaves traceable evidence in one shared repository.

Pitfalls, mistakes, and what the examiner looks for

- **Building with the wrong JDK.** The handout pins JDK 11 (IntroLab p.1); a newer default JDK is the classic silent saboteur. A good report mentions *verifying* the toolchain, not just installing it — it shows you treat reproducibility as a deliberate property.
- **Committing build output.** `target/` directories full of `.class` and `.jar` files are exactly the generated artifacts the Git deck crosses out (Git Lab p.6). An examiner reading your history sees this instantly; a clean `.gitignore` from commit one signals discipline.
- **Treating the lab as "just setup."** The exam template asks about your repository and project basis (What-To-Expect p.1). Students who can justify *why* fork-then-clone, *why* `install`, *why* pinned versions turn a trivial lab into easy why/how marks; students who write "we installed Maven and it worked" earn nothing for it.
- **One member's machine becomes the team's machine.** If only one person can build, every later lab bottlenecks. The point of the pinned toolchain and the per-member clone is that *everyone* has the full environment — decentralization in practice (Git Lab p.7, p.9).
- **No smoke test.** Running the GUI is the verification step of this mini-change. Skipping it means discovering a broken environment in a later lab, where the failure is harder to attribute.

Theory links

- **The staged life-span model.** JHotDraw arrives as software already deep in its life span; the course drops you into the **Evolution/Servicing** stages rather than initial development (Introduction p.21). The IntroLab is your entry into someone else's running system — the defining condition of maintenance work.
- **Essential difficulties.** The first build confronts **complexity** (a large multi-module codebase) and **invisibility** (nothing about the directory tree shows you what the software is — only running it does) (Introduction p.9). The GUI smoke test is a small act of making the invisible visible.
- **Decentralized version control.** Fork → clone → full local history is Git's decentralized model in action: every member holds the complete repository, and the GitHub fork is just the agreed synchronization point (Git Lab p.7, p.9, p.15).
- **Reproducible builds as a CI precondition.** Pinning Maven 3.8.x/JDK 11 anticipates the CI principle that the build server gets code from the repository and builds it identically anywhere (ContinuousIntegration p.6); the CILab later automates exactly the build this lab performs by hand (CILab p.1).
- **GitHub flow.** The one-branch-per-member rule introduced here (IntroLab p.1) is the collaboration pattern every later lab assumes, and the mechanism that keeps concurrent work isolated until integration is deliberate (Git Lab p.5, p.11).

Git Lab (Lecture 1) — branching, merging, collaboration workflow

The assignment — what the handout asks

The Git Lab is a 19-page slide deck rather than a task sheet, organised as a funnel from tool-agnostic version-control concepts to the concrete course project. The instructional content comes first: why version control

exists — six benefits: enforce discipline, archive versions, maintain historical information, enable collaboration, recover from accidental deletions or edits, conserve disk space (Git Lab p.3); the three **essential actions** Add, Commit, Update (Git Lab p.4); a two-developer **VCS workflow** synchronising `foo.txt` through a central repository (Git Lab p.5); the "**Don't do it**" warning against committing executables and large binaries (Git Lab p.6); and **centralized vs. decentralized** VCS architectures (Git Lab p.7).

Then Git specifically: a decentralized VCS by Linus Torvalds, built to handle large projects efficiently — for example Linux (Git Lab p.9); installation per platform from `git-scm.com/downloads` (Git Lab p.10); and the central **four-area workflow diagram** — workspace, index, local repository, remote — with the commands clone, add(-u), commit, push, fetch, merge, pull, diff HEAD, diff drawn as arrows between areas (Git Lab p.11). Two live demos follow: **Demo 1 Basics** on local repositories — `git status`, `git diff [files]`, `git commit -a [-m <message>]`, `git show` (Git Lab p.12–13) — and **Demo 2 Advanced** on remote repositories — `git clone <repository>`, `git pull`, `git push` (Git Lab p.14–15), then a GitHub section (Git Lab p.16) and the Projects slide naming **JHotDraw** (Git Lab p.18).

The actual assignment is the final **LAB TODOs** slide (Git Lab p.19), four tasks:

1. **Clone** the remote repository from GitHub.
2. **Get familiar with Git commands** — "see this slide for an overview," pointing back at the four-area workflow (Git Lab p.11).
3. **Commit your initial project code** using the Git commands add, commit, pull, push…
4. **Input a change request as an Issue at GitHub.**

Task 4 is easy to miss and quietly important: it is the course's first **Initiation** act — a change request entering the system through a tracker — two lectures before Initiation is formally taught (SoftwareChange p.6).

Step-by-step walkthrough — how to do it

1. Clone and identify yourself. `git clone https://github.com/<team>/JHotDraw.git` copies the entire repository — every file, every commit — and links the remote as origin (Git Lab p.15). (beyond slides — practical knowledge) Before the first commit, set your identity so the graded history attributes work correctly: `git config user.name "Your Name"` and `git config user.email you@example.com`. With multiple members committing, correct attribution is part of what makes the repository readable as evidence.

2. Internalize the four-area model. Every Git command on the workflow slide is an arrow between two of four areas (Git Lab p.11): the **workspace** (your editable files), the **index** or stage (what the next commit will contain), the **local repository** (committed history, HEAD), and the **remote repository** (the shared fork on GitHub). The command map: `add` / `add -u` moves changes workspace → index; `commit` records index → local repository; `push` publishes local repository → remote; `fetch` brings remote commits into the local repository without touching your files; `merge` integrates them into the workspace; `pull` is fetch followed by merge; `diff` compares workspace against index; `diff HEAD` compares workspace against the last commit (Git Lab p.11). Spend ten minutes deliberately exercising each arrow on a scratch file — the lab's "get familiar" task is exactly this (Git Lab p.19).

3. Run the inner loop before every commit. The Local Repositories slide prescribes the discipline: do some editing, `git status` to see *which files* changed, `git diff [files]` to see *which lines* changed, then `git commit -a -m "<message>"` to record it; `git show` displays the commit you just made (Git Lab p.13). The loop is "look before you record": status gives the scope, diff gives the content, and only then do you commit — so stray edits and leftover debug code are caught before they enter the graded history.

4. Branch per feature — GitHub flow. The IntroLab already committed the team to GitHub flow: each member works on their own feature branch (IntroLab p.1). (beyond slides — practical knowledge) The commands, which the deck does not spell out: `git checkout -b feature/<short-name>` creates and switches to the branch; work and commit there; `git push -u origin feature/<short-name>` publishes it and sets the upstream so later pushes are just `git push`. Integration happens through a **pull request** on GitHub: open a PR from the feature branch into main, let a teammate review, and merge when green. Main stays releasable at all times; unfinished work lives only on branches.

5. Write commit messages that narrate maintenance. (beyond slides — practical knowledge) The repository is graded on "overall maintenance work" (What-To-Expect p.5), and the cheapest way to make history legible is messages that state the *why*: "Extract createWatermark() from export path to localize format-independent stamping" tells a story; "fixed stuff" does not. Commit in small, logically complete increments — the index exists precisely so you can stage *part* of your edits (`git add <file>` selectively, or `git add -u` for all tracked changes) and record clean, single-purpose commits (Git Lab p.11).

6. Synchronise: pull before push. When a teammate has pushed first, your `git push` is rejected because your local repository is missing their commits. The fix is the deck's own command pair: `git pull` (fetch + merge) to integrate their work locally, resolve any conflicts, then push (Git Lab p.11, p.15). On a shared branch this is routine; under GitHub flow it is rare, because members work on disjoint branches and integrate through PRs instead of racing pushes on main.

7. Resolve a merge conflict deliberately. (beyond slides — practical knowledge) When `merge` or `pull` reports a conflict, Git stops and writes both versions into the file between `<<<<<<<`, `=======`, `>>>>>>>` markers. Open the file, decide what the combined code must be — which sometimes means neither side verbatim — remove the markers, `git add` the resolved file, and `git commit` to complete the merge. Then rebuild and rerun the application before pushing: a textually resolved conflict can still be semantically wrong, and the merge commit is part of the graded history.

8. Keep generated files out. The "Don't do it" slide bans executables and large binaries from the repository; the repository should hold the source files from which everything else is generated, not the generated local files (Git Lab p.6). For JHotDraw that means `target/` (Maven build output), IDE metadata, and OS junk belong in `.gitignore` (beyond slides — practical knowledge). The rationale on the slide is structural, not cosmetic: a VCS archives versions efficiently as deltas of text; binaries defeat the delta storage and bloat every clone.

9. File the change request as a GitHub Issue. The final TODO (Git Lab p.19): on the fork's Issues tab, create a new Issue describing a change — a feature, a bug fix, or an improvement, the three change-request kinds the ChangeReqLab names (ChangeReqLab p.1). (beyond slides — practical knowledge) Give it a behavior-focused title, describe current vs. desired behavior in the body, and label it. This Issue is the seed the next lab's user story refines, and the number it gets can be referenced from commits ("see #12") to weave code and request together in the graded record.

10. Verify the published state. Browse the fork on GitHub and check what an examiner would see: the branch list, the commit history with sensible messages, the Issue, and no build output anywhere. The deck's GitHub section exists precisely because the hosted view — forks, Issues, pull requests — is where the collaboration features live on top of bare Git (Git Lab p.16).

Why & how — report-ready justifications

- I worked on a **feature branch per change instead of committing directly to main**, because branch isolation keeps concurrent teammates' work from colliding line-by-line and keeps main releasable

at every moment, whereas shared-main development would have made every push a potential integration emergency (IntroLab p.1; Git Lab p.5, p.11).

- I committed in **small, single-purpose increments rather than end-of-day dumps**, because small commits make the history a readable narrative of the maintenance process and let any single step be reverted without dragging unrelated work with it — and the index exists precisely to compose such commits deliberately (Git Lab p.11, p.13).
- I ran `git status` **and** `git diff` **before every commit instead of committing blind**, because inspecting scope and content first catches stray edits and debug leftovers before they pollute the permanent record, at the cost of seconds (Git Lab p.13).
- I **pulled before pushing rather than forcing my version**, because pull integrates teammates' published commits into my local repository and surfaces conflicts on my machine — where I can build and test the resolution — instead of breaking the shared remote (Git Lab p.11, p.15).
- I kept **build output out of version control via** `.gitignore` **instead of committing** `target/`, because the repository should archive the human-authored sources from which artifacts are generated; committing binaries defeats delta storage, bloats every clone, and buries the meaningful diff (Git Lab p.6).
- I recorded the change request as a **GitHub Issue rather than an email or verbal agreement**, because the tracker gives the request an identity, a discussion thread, and a link target for commits and pull requests, turning Initiation into a traceable artifact instead of folklore (Git Lab p.19).
- We used **Git, a decentralized VCS, rather than a centralized one**, because every member holds the full history locally — commits, diffs, and branches work offline and fast — and the central GitHub fork is a coordination convention, not a single point of failure (Git Lab p.7, p.9).
- I wrote commit messages that **explain why the change was made rather than restating what the diff shows**, because the diff already answers "what" — the message is the only place the reasoning survives, and the graded repository is read as evidence of disciplined maintenance (Git Lab p.13; What-To-Expect p.5).

Copy-paste reflection for the report

In the Git lab I set up the version-control discipline that carried all my later course work. I cloned our team's JHotDraw fork, which gave me the complete history locally and linked the team repository as origin (Git Lab p.15), and I practised the four-area workflow — workspace, index, local repository, remote — until the command map was second nature: add to stage, commit to record locally, push to publish, pull to integrate (Git Lab p.11). For every subsequent change I followed GitHub flow: I created a feature branch [branch name] for [change], committed in small steps with messages explaining why — for example "[commit message]" — and merged through a pull request reviewed by [teammate]. Before each commit I ran git status and git diff so I knew exactly what was entering the history (Git Lab p.13), and I kept generated files out of the repository with a .gitignore covering target/, following the deck's rule that the repository holds sources, not build output (Git Lab p.6). I also filed my change request as GitHub Issue [#N], which later anchored my user story and let my commits reference the request directly (Git Lab p.19). The payoff came when [incident — e.g. a refactoring went wrong / two of us edited the same class]: because the work was isolated on branches and recorded in small commits, I could [revert/diff/merge] in minutes instead of reconstructing work, and main never stopped building.

Likely exam questions about this lab — with answers

Q: What is the difference between `commit` **and** `push`**?** `commit` records the staged changes into the *local* repository — it is an offline operation that creates history only on my machine (Git Lab p.11). `push` publishes those local commits to the *remote* repository (origin, our GitHub fork) so teammates can see and fetch them

(Git Lab p.15). The separation is decentralization in action: I can commit freely while offline and choose when to publish. Teammates see nothing until I push.

Q: What is the difference between `pull` and `fetch`? `fetch` transports new commits from the remote into my local repository but leaves my workspace untouched; `merge` then integrates them into my working files. `pull` is the two combined: fetch followed by merge (Git Lab p.11). Fetch alone is useful when I want to inspect incoming changes before integrating them; pull is the everyday shortcut when I am ready to synchronise.

Q: What is the difference between `diff` and `diff HEAD`? Both compare from the workspace, but to different baselines: `diff` compares workspace against the **index**, showing only unstaged edits, while `diff HEAD` compares workspace against the **last commit**, showing all changes since HEAD — staged and unstaged together (Git Lab p.11). After `git add -u`, plain `diff` shows nothing while `diff HEAD` still shows the edits. Knowing the difference tells you exactly what the next commit will contain.

Q: Why should executables and large binaries not be committed? The repository should hold the human-authored source files from which everything else is generated, not the generated outputs (Git Lab p.6). Binaries defeat the VCS's efficient delta-based storage — every version is stored nearly whole — so the repository bloats and every clone slows down, which works against the "conserve disk space" benefit version control is supposed to provide (Git Lab p.3). Generated files also produce meaningless diffs, drowning the history's signal. The correct home for build artifacts is the build itself, reproducible from source on demand.

Q: What is the difference between centralized and decentralized version control, and which is Git? In a centralized VCS, one server holds the repository and developers hold only working copies, so history operations require the server. In a decentralized VCS every developer's copy is a full repository with complete history that synchronises with peers (Git Lab p.7). Git is decentralized — created by Linus Torvalds to manage large projects like Linux efficiently (Git Lab p.9). Practically this means commits, diffs, and branches are local and fast, and the team's GitHub fork is a convention for coordination rather than a technical necessity.

Q: How did you structure your branches and why? We followed GitHub flow: main holds only integrated, building code, and each member develops on their own feature branch, merged back through pull requests (IntroLab p.1). I chose this over shared-main development because isolation keeps half-finished work from breaking teammates, and over long-lived personal branches because frequent small merges keep integration pain proportional to the change. The PR step adds review before integration — and it is also what our CI pipeline later hooked into, building every pull request automatically (CILab p.1).

Q: Why is the GitHub repository part of your exam grade, and what in it shows maintenance work? The examiner evaluates the repository alongside the report: code changes, refactoring work, testing, pipeline setup, and overall maintenance work (What-To-Expect p.5). A disciplined history demonstrates the process itself — feature branches show isolated changes, small commits with reasoned messages narrate each change's why, Issues record Initiation, and merged PRs with green builds show verified integration. The report references this evidence; the repository proves the report's claims are real.

Pitfalls, mistakes, and what the examiner looks for

- **Confusing commit with push** — claiming work was "shared" when it was only committed locally. The four-area model is the examiner-visible mental model; getting it wrong in the report undermines everything else you say about Git (Git Lab p.11).
- **The everything-commit.** One commit per day titled "work" makes the history useless as evidence. The graded artifact is the *narrative* — small commits, each with a why (What-To-Expect p.5).

- **Committing `target/`, `.class` files, or IDE folders.** Directly violates the deck's "Don't do it" slide (Git Lab p.6) and is visible to the examiner in seconds.
- **Branching theatre.** Creating a feature branch but merging it unreviewed two minutes later, or doing all real work on main anyway. The examiner can read the network graph; the report's claims about GitHub flow must match it.
- **Forgetting the Issue.** The fourth LAB TODO — input a change request as a GitHub Issue (Git Lab p.19) — is the hook the whole change process hangs on. A repository with no Issues has no recorded Initiation.
- **What distinguishes a good report answer:** naming the four areas precisely, quoting your own branch names and commit messages, and tying each Git practice to a maintenance benefit (recoverability, isolation, traceability) rather than describing Git generically. Specific beats generic: "my history shows X" outscores "Git allows X" (What-To-Expect p.1, p.5).

Theory links

- **Why VCS at all.** The six benefits — discipline, archived versions, history, collaboration, recovery, disk efficiency (Git Lab p.3) — are the lab's answer to the essential difficulty of **changeability**: software is changed constantly, and version control is what makes constant change survivable (Introduction p.9).
- **Essential actions and workflow.** Add/Commit/Update (Git Lab p.4) and the two-developer `foo.txt` synchronisation (Git Lab p.5) are the minimal theory of collaborative change; Git's four areas refine the same picture with a staging index and a local/remote split (Git Lab p.11).
- **Conclusion phase of the change mini-cycle.** Committing verified work and making it the new baseline is exactly the **Conclusion** of a software change (SoftwareChange p.14); every later lab ends by traversing this lab's command map.
- **Initiation via Issues.** The change-request Issue (Git Lab p.19) is the first concrete **Initiation** artifact, formalized next lecture as user stories in the product backlog (ChangeInitiation p.5, p.7; ChangeReqLab p.1).
- **Versioned staged model.** Parallel supported versions in the versioned staged life-span model correspond to parallel branches — the branching introduced here is the mechanism that model presupposes (Introduction p.23).
- **CI's foundation.** "Maintain a code repository" is the first CI principle; the pipeline lab builds directly on this lab's repository and PR discipline (ContinuousIntegration p.3; CILab p.1).

Change Request Lab (Lecture 2) — user stories and the GitHub backlog

The assignment — what the handout asks

The ChangeReqLab ("CASE Study Lab") is a one-page handout. Its introduction states the premise: "Software changes are started by creating a change request. A change request can be a new feature, a bug fix and improvements. User stories are short and simple descriptions of features written from the perspective of the person who desires the new capability" (ChangeReqLab p.1). The two objectives: write a user story for an **existing** software feature, and select a user story for "your mandatory individual portfolio assignment" (ChangeReqLab p.1).

The classwork is five tasks in order (ChangeReqLab p.1):

1. Each **team** creates a GitHub **fork** of the project repository [JHotDraw] (already done if the IntroLab fork stands; restated here because this lab can be a fresh start).
2. In future lab exercises, each team member follows **GitHub flow** to collaborate.

3. **Select an existing feature in JHotDraw and write the user story** — the handout links a list of [existing features], so the story is reverse-engineered from behavior that already exists.
4. Use **GitHub Projects** — the course's backlog tool.
5. **Create a Card for your User Story and put it in the TODO Backlog.**

The portfolio work: "Write the User Story for your selected JHotDraw feature. Note this is an artifact for your portfolio" (ChangeReqLab p.1). Two scoping details matter: the fork is per-team, but the story — and the portfolio assignment it anchors — is explicitly **individual**; and the story describes an *existing* feature, which forces you to observe real behavior and recover the who/what/why behind it rather than inventing requirements.

Step-by-step walkthrough — how to do it

1. Choose a feature you can observe. Run the SVG editor (IntroLab p.1) and work from behavior, not folklore. The demo drawing itself is a feature menu: **connections** (figures joined by lines), **groups**, **text**, **annotation and connected text**, **images**, and **URL attachments** are all labelled on the deck's annotated screenshot (JHotDraw p.4, p.6). Pick a feature whose behavior you can trigger reliably with the mouse — you will need to trigger it again under the debugger in the next lab (CLLab p.1), so a feature you cannot demonstrate is a feature you cannot locate.

2. Identify the stakeholder. A user story is written "from the perspective of the person who desires the new capability" (ChangeReqLab p.1; ChangeInitiation p.5). For an existing feature, ask who benefits from it as shipped: a diagram author, a presenter exporting drawings, a developer embedding the framework. The stakeholder choice determines the story's *so that* clause — the motivation — and later the acceptance criterion.

3. Write the story in the canonical shape. "As a `<user type>`, I want `<goal>` so that `<reason>`" (ChangeInitiation p.4–6). The course's model is the watermark request rewritten as: "As a **trial-version user**, I want **exported drawings to automatically carry a watermark text** so that **the demo origin is visible in every export format**" (ConceptLocation p.6; ChangeInitiation p.4–5). Imitate the anatomy: a concrete user type, an observable capability, a reason that explains value rather than restating the goal.

4. Apply the card test and split if needed. A story should fit a 3"×5" card; if it does not, split it into smaller stories (ChangeInitiation p.4). A story about "exporting" that covers three formats and a preferences dialog is several stories. Splitting is not bureaucracy — each story should carry one testable intention, because the story's nouns become the next lab's search concepts and its *so that* clause eventually becomes a verification scenario.

5. Extract the domain concepts while you are at it. (beyond slides — practical knowledge) Underline the story's nouns the way the lecture underlines the watermark request's — Export, Drawing, Text, Format (ConceptLocation p.6). Writing the concept list next to the story now saves the first ten minutes of the Concept Location lab and proves you understand the story's role as input to the next phase.

6. Set up the GitHub Projects board. On the team fork, create a Project (the Projects tab) and give it at least a **TODO** column — a Kanban-style backlog (ChangeReqLab p.1). (beyond slides — practical knowledge) Typical columns: TODO / In progress / Done; the lab only requires that your story's card starts in TODO, which realizes the lecture's "add the change request to the Product Backlog" (ChangeInitiation p.7–8).

7. Create the card — and link it to the Issue. Create a card containing your user story and place it in the TODO column (ChangeReqLab p.1). (beyond slides — practical knowledge) If you filed a change-request Issue in the Git Lab (Git Lab p.19), convert the card to that Issue or reference it, so the backlog card, the tracker

entry, and later the feature branch and pull request form one traceable chain: request → backlog → branch → commits → merge.

8. Record the story as a portfolio artifact. Copy the final story — user type, goal, reason, plus your concept list and any acceptance note — into your portfolio document. This exact text is what the exam report's Initiation questions will be answered from, and the examiner can check it against the card on the graded repository (What-To-Expect p.1, p.5).

Why & how — report-ready justifications

- I expressed the change request as a **user story rather than a free-form description**, because the fixed As-a/I-want/so-that shape forces the request to name its stakeholder, capability, and value in one testable sentence, while free-form prose lets the *who* and the *why* silently go missing (ChangeReqLab p.1; ChangeInitiation p.4–6).
- I wrote the story about an **existing feature rather than an invented one**, because reverse-engineering a story from observable behavior trains the perspective-taking the format demands and guarantees the feature is real, demonstrable, and locatable in the code in the next lab (ChangeReqLab p.1; CLLab p.1).
- I kept the story **small enough for a 3×5 card instead of letting it cover the whole feature area**, because one story should carry one testable intention; an oversized story would have blurred concept location and made the eventual impact estimate meaningless (ChangeInitiation p.4).
- I phrased the story in **domain language rather than solution language**, because naming concepts like [drawing, export, figure] rather than classes and methods keeps Initiation independent of the code and gives Concept Location clean search terms instead of premature design decisions (ConceptLocation p.6).
- I put the story on a **GitHub Projects card in the TODO backlog rather than in a private note**, because the backlog makes the request visible, prioritizable, and linkable to Issues, branches, and pull requests — turning my individual assignment into a traceable part of the team's process (ChangeReqLab p.1; ChangeInitiation p.7–8).

Copy-paste reflection for the report

In the change-request lab I initiated my course change the way the lecture's change process begins: with a change request written as a user story. I selected the existing JHotDraw feature [feature, e.g. grouping of figures / export / connections] after exercising it in the running SVG editor, and wrote: "As a [user type], I want [capability] so that [reason]." I deliberately wrote from the stakeholder's perspective rather than my own as developer, because the story format exists to record who needs the capability and why — not how it will be coded (ChangeReqLab p.1). My first draft was too broad, covering both [aspect A] and [aspect B], so I applied the card-size rule and split it, keeping [aspect A] as my story (ChangeInitiation p.4). I underlined the story's domain concepts — [concept 1], [concept 2], [concept 3] — which a week later became my debugger search targets in concept location, exactly as the lecture's watermark example extracts Export, Drawing, Text and Format from the request text (ConceptLocation p.6). Finally I created a card for the story on our GitHub Projects board and placed it in the TODO backlog, linked to Issue [#N], so the request had a visible, prioritizable identity in the team's process (ChangeReqLab p.1). Looking back, the discipline of writing the story before touching code is what kept my later work scoped: every phase that followed — location, impact analysis, the change itself — answered to one sentence.

Likely exam questions about this lab — with answers

Q: What is a change request, and what forms can it take? A change request is the artifact that starts a software change: a recorded statement of a desired difference in the system. The handout names three kinds — a new feature, a bug fix, and improvements (ChangeReqLab p.1). In our process it enters as a GitHub Issue and a backlog card, and the change mini-cycle's Initiation phase is precisely the creation and prioritization of such requests (SoftwareChange p.6; ChangeInitiation p.7).

Q: What is a user story, and what are its parts? A user story is a short, simple description of a feature written from the perspective of the person who desires the capability (ChangeReqLab p.1; ChangeInitiation p.5). Its canonical shape has three slots: "As a `<user type>`" names the stakeholder, "I want `<goal>`" names the observable capability, and "so that `<reason>`" names the value. It should fit a 3×5 card; if it cannot, it should be split into smaller stories (ChangeInitiation p.4).

Q: Why write a user story for a feature that already exists? Because the lab's learning goal is the *format and perspective*, not the invention of requirements: recovering the who/what/why from observable behavior exercises exactly the stakeholder thinking the story format demands (ChangeReqLab p.1). It also guarantees the story is grounded — the feature can be demonstrated, debugged, and located in code in the following labs, which an invented capability might not allow (CLLab p.1).

Q: How does the user story drive the rest of the change process? The story's nouns are the domain concepts that Concept Location searches for in the code — the lecture's watermark request yields Export, Drawing, Text, Format (ConceptLocation p.6). The located classes seed Impact Analysis (SoftwareChange p.9). And the story's *so that* clause implies the acceptance criterion that verification eventually checks. One sentence therefore travels through the whole mini-cycle, which is why getting it small and precise matters so much at the start.

Q: What is the product backlog, and how did you realize it in this course? The product backlog is the prioritized collection of pending change requests from which work is drawn (ChangeInitiation p.7). We realized it with GitHub Projects on our team fork: each user story is a card, new stories enter the TODO column, and cards move across the board as work proceeds (ChangeReqLab p.1). Realizing it in the same platform as the code means a card links to the Issue, branch, and pull request that implement it — the backlog and the evidence stay connected.

Q: Why is the user story an individual portfolio artifact when the fork is per-team? The course separates shared infrastructure from individual assessment: the team shares one fork, board, and workflow, but each member selects their own feature, writes their own story, and carries it through the subsequent phases as their mandatory individual portfolio assignment (ChangeReqLab p.1). The exam grades individuals, so each student needs an end-to-end change of their own to reflect on (What-To-Expect p.1).

Pitfalls, mistakes, and what the examiner looks for

- **Solution-language stories.** "As a developer, I want a WatermarkDecorator class…" is a design masquerading as a request. The story must state stakeholder value; the design comes phases later. Examiners read this instantly as a process misunderstanding.
- **The bloated story.** A story that needs a paragraph fails the card test and should have been split (ChangeInitiation p.4). In the report, narrating the split you performed is worth more than pretending the first draft was perfect.
- **A missing or circular *so that*.** "…so that drawings are exported" restates the goal instead of giving a reason. The *so that* clause is where the stakeholder's value lives — and where your acceptance criterion will come from.

- **Backlog as decoration.** Creating the card after the work is done inverts the process. The card must enter TODO at Initiation; its later movement across the board is part of the repository's graded story (What-To-Expect p.5).
- **What distinguishes a good report answer:** quoting your actual story verbatim, naming the feature and stakeholder, explaining one concrete formatting decision (the split, the stakeholder choice, the wording of the reason), and tracing the story's nouns into your concept location. The story is small; the reflection on it should show the machinery behind every word (What-To-Expect p.1).

Theory links

- **Initiation phase.** This lab is the course's hands-on **Initiation**: a change request is created, expressed, and entered into the backlog — phase one of the change mini-cycle (SoftwareChange p.6; ChangeInitiation p.5, p.7).
- **User stories as requirements in the agile paradigm.** The story-card-backlog apparatus is the agile loop's intake: Backlog → Iteration → Release, with stories as the unit of planning (Introduction p.17–18; ChangeInitiation p.7–8).
- **The watermark exemplar.** The lecture's dissected change request — three sentences yielding four underlined concepts — is the model for treating request text as the input to Concept Location (ConceptLocation p.6).
- **Evolution in the staged model.** Feature requests against a living system are what the Evolution stage consists of; each story is one increment of evolution entering the pipeline (Introduction p.21).
- **Forward traceability.** Story → Issue → card → branch → PR is the traceability chain that lets the examiner (and any maintainer) connect a requirement to the commits that realize it — the practical answer to software's invisibility (Introduction p.9; What-To-Expect p.5).

Concept Location Lab (Lecture 2) — locating the code with the IDE debugger

The assignment — what the handout asks

The CLLab is a one-page handout whose introduction supplies a definition you should be able to quote: "Dynamic program analysis is the analysis of computer software that is performed by executing programs on a real or virtual processor. For dynamic program analysis to be effective, the target program must be executed with sufficient test inputs to produce interesting behavior. Use of software testing measures such as code coverage helps ensure that an adequate slice of the program's set of possible behaviors has been observed" (CLLab p.1).

The two objectives (CLLab p.1):

1. **Apply the IDE Debugger to locate feature concepts at runtime.**
2. **Create a list of the initial set of classes** from the concept-location results.

The classwork is one composite task with two operational hints baked in (CLLab p.1):

- "Use the IDE Debugger to localize the classes that involves your Change Request."
- Hint one: "Features are often started from the **controller classes**."
- Hint two, the scoping rule: "if your feature involves a large number of classes then localize **only the domain classes** that are related to the domain concepts."

The portfolio work: "Write the initial set of classes in **table format** as a result of your concept location based on your selected feature," with exactly two columns — **Domain Class | Responsibility** (CLLab p.1). That table is the lab's gradable artifact and, in process terms, the **initial impact set** the next lab consumes (SoftwareChange p.9).

Step-by-step walkthrough — how to do it

1. Put the change request on the desk and extract its concepts. Concept location "finds the code snippet where a change is to be made," and its input is the change request's **domain concepts** — change requests are formulated in the vocabulary of users, not of the code (ConceptLocation p.2). Underline the domain nouns in your user story the way the lecture dissects the watermark request: "Modify the **export** feature of JHotDraw to automatically include a simple watermark **text** in the **drawings** being exported … uniformly for all possible export **formats**" yields **Export, Drawing, Text, Format** (ConceptLocation p.6). Your concept list is your search agenda; everything that follows is finding where each concept lives.

2. Seed candidates with grep — knowing its limits. (beyond slides — practical knowledge, though the technique itself is lectured:) a quick pattern-matching pass over the source narrows thousands of files to a shortlist. GREP — "global regular expression print" — prints the lines matching a regular expression, used iteratively: formulate a query, investigate results, refine (ConceptLocation p.8).

```
# from the JHotDraw source root — iterative query refinement
grep -rn "export" --include="*.java" | less     # too broad? refine:
grep -rln "OutputFormat" --include="*.java"
grep -rn "class .*OutputFormat" --include="*.java"
```

Treat hits as candidates only. Grep searches the **Name** corner of the concept triangle, and names are unstable — the same concept can be called Dog/Pes/Hund (ConceptLocation p.4–5). The code that implements "export" may be named `StorageFormat` or `ImageWriter`. The reliable edge is **Intension → Extension**: hold the concept's meaning and *recognize* the code that implements it, whatever it is called (ConceptLocation p.4).

3. Find the controller entry point. The handout's hint — features often start from controller classes (CLLab p.1) — is precise in JHotDraw: the framework is MVC, and the **Tool hierarchy is the controller**; toolbar- and menu-triggered behavior enters through a `Tool` or an `Action` class (JHotDraw p.17–18). Run the SVG editor (`mvn exec:java "-Dexec.mainClass=org.jhotdraw.samples.svg.Main"` from `jhotdraw-samples-misc`, IntroLab p.1) and note exactly which gesture triggers your feature — that gesture's handler is where execution enters the feature.

4. Set breakpoints and run under the debugger. (beyond slides — practical knowledge) Open the project in the IDE (NetBeans/IntelliJ/Eclipse — any IDE debugger satisfies the handout), set a breakpoint in the candidate entry class from steps 2–3, and start the SVG `Main` class in **Debug** mode. Then trigger the feature in the GUI:

```
1. Breakpoint in the suspected entry point (e.g. the export Action's actionPerformed,
   or the Tool's mouse handler for canvas features).
2. Trigger the feature with a real input (an actual drawing, an actual export).
3. When the breakpoint hits, read the CALL STACK top to bottom — it is a ready-made
   list of the classes the feature flows through, in order.
4. Step Into (F7/F5) calls that carry the concept; Step Over (F8/F6) framework
   plumbing (event dispatch, repaint, listeners).
5. At each frame, note the class and what it contributes; follow the data
   (the Drawing, the figures, the output stream) rather than the control noise.
```

The handout's effectiveness condition governs the inputs: dynamic analysis only reveals behavior you actually execute, so exercise the feature with **sufficient test inputs to produce interesting behavior** — for an export feature, export more than one drawing and more than one format; the code-coverage idea is the guard against concluding from one unrepresentative run (CLLab p.1).

5. Decide "implemented here" vs. "flows through here" — the dependency-search judgment. The debugger shows you every class the feature *touches*; the lecture's dependency-search machinery tells you which ones to *keep*. For each class on the stack ask the two questions in the algorithm's order: is the concept implemented in the **module itself** (its **local functionality** — what it does without delegating)? If yes, stop — located. If no: is it in the **composite functionality** (what it delivers through its suppliers)? If yes, descend into the supplier; if no, backtrack (ConceptLocation p.9–10). An export menu action that merely delegates to a format writer has Export in its composite but not its local functionality — it is a conduit, not the location. Track your progress with the four marks: **Blank** (never inspected, not scheduled), **Next** (scheduled), **Propagating** (composite contains the concept — live path), **Unchanged** (composite lacks it — dead end) (ConceptLocation p.11). Expect at least one wrong turn; the lecture's own UMLEditor trace deliberately walks into `NoteNode`, marks it Unchanged, and backtracks before finding `ClassNode` (ConceptLocation p.16–18).

6. Apply the scoping rule. If the trace involves many classes, record **only the domain classes related to the domain concepts** (CLLab p.1) — the classes that mean something in the request's vocabulary (drawing, figure, format, export), not the Swing event plumbing the stack is full of. This is the lab enforcing **partial comprehension**: large programs cannot be completely understood, so you build the minimum essential understanding as-needed — learn the route to your destination, not the whole city (ConceptLocation p.3).

7. Write the Domain Class / Responsibility table. The portfolio artifact, in the handout's exact two-column format (CLLab p.1). Shape (illustrative content for a watermark-style change — substitute your feature's classes):

Domain Class	Responsibility
SVGDrawingPanel	Hosts the drawing view; entry point of the export action
Drawing	The domain object being exported; owns the figures
ImageOutputFormat	Writes the drawing to a raster image file — candidate change location
SVGOutputFormat	Writes the drawing to SVG — second format the change must cover

(beyond slides — practical knowledge) Add the table to your portfolio document and commit it; reference the change-request Issue in the commit message so the located classes are traceably tied to the request they answer.

8. Sanity-check against the story. Every concept you underlined in step 1 should be accounted for by at least one row, and every row should justify itself in concept language. If a concept has no class, you have not finished locating; if a class has no concept, it probably belongs to the plumbing you should have scoped out.

Why & how — report-ready justifications

- I located the concept with **dynamic analysis under the IDE debugger rather than by reading the source top-down**, because the running program shows which classes demonstrably participate in the feature, while reading alone cannot tell live code from dead and would have meant comprehending far more of JHotDraw than the change needed (CLLab p.1; ConceptLocation p.3, p.7).

- I started from the **controller classes rather than from the domain model**, because features enter through user gestures and JHotDraw's Tool/Action hierarchy is the MVC controller — the one place a feature's execution is guaranteed to pass — whereas guessing at model classes by name would have relied on the unstable Name edge of the concept triangle (CLLab p.1; JHotDraw p.17–18; ConceptLocation p.4).
- I used **grep only to seed candidates, not to conclude**, because pattern matching finds where a word appears, not where a concept is implemented — it is blind to renaming and to delegation — so every grep hit was verified in the debugger or by reading the class's actual responsibility (ConceptLocation p.8, p.4–5).
- I distinguished classes the feature **flows through from classes that implement it**, using the local-versus-composite functionality test, because stopping at a delegating conduit would have put my change in a class that merely forwards the work, while the real implementation lived one supplier deeper (ConceptLocation p.9–10).
- I exercised the feature with **several distinct inputs rather than one**, because dynamic analysis is only as informative as the behaviors actually executed — the handout's own effectiveness condition — and a single run would have hidden the format-dependent paths my change must cover (CLLab p.1).
- I recorded **only the domain classes, not every executed class**, because the lab's scoping rule operationalizes partial comprehension: the deliverable is the minimum set of classes that carry the request's concepts, which kept the initial impact set small enough for the next phase to inspect honestly (CLLab p.1; ConceptLocation p.3).
- I wrote the result as a **Domain Class / Responsibility table rather than prose notes**, because the table is the handoff artifact: each row pairs a location with the reason it matters, which is exactly the seed Impact Analysis marks CHANGED on day one of the next lab (CLLab p.1; SoftwareChange p.9).

Copy-paste reflection for the report

In the concept-location lab I found where my change request lands in JHotDraw's code. I started from my user story and extracted its domain concepts — [concept 1], [concept 2], [concept 3] — following the lecture's watermark example, where the request text yields Export, Drawing, Text and Format (ConceptLocation p.6). A first grep pass over the source gave me candidate classes, but I treated those hits only as seeds, because pattern matching finds names, not meanings, and the code's vocabulary differs from the request's (ConceptLocation p.8). The real location work was dynamic: I set a breakpoint in [entry class] — found via the handout's hint that features start from controller classes, which in JHotDraw means the Tool/Action hierarchy (CLLab p.1; JHotDraw p.17) — ran the SVG editor in debug mode, and triggered [feature] with [inputs]. Reading the call stack and stepping into the calls that carried the concept, I separated classes the feature merely flows through from the class that actually implements it, applying the local-versus-composite functionality test from dependency search (ConceptLocation p.9). One path was a dead end: [class] turned out to [reason], so I marked it unchanged and backtracked, exactly like the lecture's UMLEditor trace (ConceptLocation p.16–17). My result is the initial set of [N] domain classes, recorded as the lab's Domain Class / Responsibility table: [Class A — responsibility], [Class B — responsibility]. That table became the initial impact set my impact analysis started from, and the discipline of partial comprehension meant I read only the slice of JHotDraw my change required (ConceptLocation p.3).

Likely exam questions about this lab — with answers

Q: What is concept location and why is it necessary? Concept location finds the code snippet where a change is to be made (ConceptLocation p.2). It is necessary because change requests are written in domain vocabulary while the implementing code may be scattered, renamed, or buried — on an unfamiliar codebase you cannot even begin a change without a deliberate search phase. Its output, the located classes, is the

starting point of the software change and the initial impact set that Impact Analysis grows (SoftwareChange p.8–9).

Q: What is dynamic program analysis, and what makes it effective? It is analysis performed by executing the program on a real or virtual processor and observing it, rather than by reading the source (CLLab p.1). It is only effective if the program is executed with sufficient test inputs to produce interesting behavior — code that is never triggered is invisible to it — and coverage measures help ensure an adequate slice of possible behaviors was observed (CLLab p.1). In the lab this meant exercising my feature with multiple realistic inputs before trusting the trace.

Q: Why does the handout say features are often started from controller classes? Because a feature begins with a user gesture, and in an MVC architecture the controller is the component that receives gestures and translates them into operations on the model. In JHotDraw the Tool hierarchy is the controller, so a canvas feature enters through a Tool and a menu feature through an Action (JHotDraw p.17–18). Breakpointing the controller therefore guarantees the debugger catches the feature's execution at its entry, from which the call stack reveals the rest of the path.

Q: Compare grep-based and dependency-based concept location. Grep is static pattern matching: it prints lines matching a regular expression, used iteratively with refined queries (ConceptLocation p.8). It is fast and needs no build, but it searches the Name corner of the concept triangle and fails when the code names the concept differently. Dependency search navigates the class dependency graph extracted from the code, asking at each class whether the concept is in its local or composite functionality — it reasons from meaning to implementation, at the cost of more judgment per step (ConceptLocation p.9–10). In practice grep seeds the search and dependency reasoning (or the debugger) finishes it.

Q: What is the difference between local and composite functionality, and why does the search stop only on local? Local functionality is what a module implements itself, without delegation; composite functionality is everything it delivers together with its suppliers (ConceptLocation p.9). A class with the concept only in its composite functionality is a conduit — the real implementation is in a supplier — so changing it would be changing the wrong place. The algorithm therefore asks "implemented in the module?" first and stops only on yes; a composite-only yes sends the search down into suppliers (ConceptLocation p.10).

Q: Which marks does the search use, and what does each mean? Blank — never inspected and not scheduled; Next — scheduled for inspection; Propagating — inspected, composite responsibility contains the concept, so the search descends through it; Unchanged — inspected, composite responsibility does not contain it, a dead end to backtrack from (ConceptLocation p.11). The marks are the algorithm's memory: they prevent re-inspecting judged classes and losing queued candidates, and they make the search reproducible — the lecture's UMLEditor trace shows them driving a wrong turn at NoteNode and a recovery to ClassNode (ConceptLocation p.12–18).

Q: What did this lab produce, and what consumes it? A table of the initial set of classes — Domain Class and Responsibility — for my selected feature (CLLab p.1). Process-wise, this is the initial impact set: Impact Analysis seeds its marking algorithm with exactly these classes and grows them into the estimated impact set along the interaction graph (SoftwareChange p.9; ImpactAnalysis p.4). A wrong or bloated table here propagates error into every later phase, which is why the lab insists on domain classes only.

Pitfalls, mistakes, and what the examiner looks for

- **One run, one input.** The handout's effectiveness condition is the first thing graders know about dynamic analysis (CLLab p.1). A trace from a single trivial input misses format-dependent and conditional paths; the report should name the inputs you used and why they were sufficient.

- **Recording the plumbing.** A debugger stack in a Swing app is mostly event dispatch and repaint. Copying every frame into the table ignores the scoping rule — domain classes only (CLLab p.1) — and signals you could not tell domain from framework.
- **Stopping at the conduit.** The menu action that *triggers* export is not the class that *implements* export. Mistaking flows-through for implemented-in is the local/composite confusion the dependency-search algorithm exists to prevent (ConceptLocation p.9–10).
- **Name-worship.** Concluding from grep hits alone trusts the Name edge of the concept triangle, the precise trap the Dog/Pes/Hund slide warns about (ConceptLocation p.4–5). The examiner expects you to say how you verified a hit was a real location.
- **A table without responsibilities.** Half the artifact is the Responsibility column — it proves you understood *why* each class is in the set, and it is the comment material Impact Analysis's Table 1 will ask for again (CLLab p.1; AnalysisLab p.2).
- **No wrong turns admitted.** A search narrative with zero backtracking reads as reconstructed after the fact. The lecture's own canonical trace includes a wrong way and a backtrack (ConceptLocation p.16–17); narrating yours is evidence of a real search, and it costs nothing.
- **What distinguishes a good report answer:** naming the methodology (dynamic search with the IDE debugger, seeded by grep, judged by local/composite), quoting your actual entry point and breakpoint, reproducing your table, and connecting it forward — "these classes became my initial impact set" (SoftwareChange p.9).

Theory links

- **Concept Location phase.** This lab is the hands-on second phase of the change mini-cycle: concepts are extracted from the change request and located in the code as the starting point of the change (SoftwareChange p.8; ConceptLocation p.2).
- **Partial comprehension / as-needed strategy.** The scoping rule (domain classes only) enacts the lecture's claim that large programs cannot be completely comprehended, so understanding is built as-needed — the visiting-a-large-city analogy (ConceptLocation p.3; CLLab p.1).
- **The concept triangle.** Name/Intension/Extension explains both why grep fails (names are unstable) and what the debugger plus human judgment do instead: recognition and location along the Intension → Extension edge (ConceptLocation p.4–5).
- **The four methodologies.** Human knowledge, traceability tools, dynamic search, static search (dependency search, pattern matching, information retrieval) — the lab implements dynamic search and borrows dependency search's decision machinery (ConceptLocation p.7).
- **Dependency search algorithm and marks.** Local vs. composite functionality, the two-question loop, Blank/Next/Propagating/Unchanged, and the UMLEditor worked trace with wrong turn and backtrack (ConceptLocation p.9–20).
- **MVC in JHotDraw.** The controller hint is grounded in the framework's architecture: Tools are the `<<controller>>` of JHotDraw's MVC, which is why they are the reliable feature entry points (JHotDraw p.17–18).
- **Handoff to Impact Analysis.** The lab's table is the initial impact set; the next lab's marking algorithm seeds CHANGED with exactly these classes (SoftwareChange p.9; ImpactAnalysis p.4).

Impact Analysis Lab (Lecture 3) — growing the initial set into the estimated impact set

The assignment — what the handout asks

The AnalysisLab is a two-page handout. Its introduction gives the quotable textbook definition: change impact analysis "can be defined as 'identifying the potential consequences of a change, or estimating what needs to be modified to accomplish a change'" (AnalysisLab p.1).

One objective (AnalysisLab p.1):

- "Apply **static and dynamic analysis** to find the **estimated impacted set of classes** based on your Change request."

One classwork task (AnalysisLab p.1):

- "Find The Estimated Impact Set of classes by following the activities illustrated in **Figure 7.9 in [Raj13]**" — Rajlich's impact-analysis marking algorithm, the same procedure the ImpactAnalysis deck draws as its activity diagrams (ImpactAnalysis p.12, p.16).

The portfolio work (AnalysisLab p.1–2): "Use Table 1 to list the **packages** and the **number of classes you visited** after you located the concept. Write short **comments** explaining what you have learned about each package and **how they contribute to your feature**." Table 1's columns are **Package name | # of classes | Comments** (AnalysisLab p.2). Note the unit shift from the previous lab: concept location reported *classes*; this lab's artifact aggregates the walk by *package*, with a learning comment per package.

Step-by-step walkthrough — how to do it

1. Start from the initial impact set. The input is the Domain Class / Responsibility table from the Concept Location Lab — process-wise, "the classes identified in concept location are the initial impact set" (ImpactAnalysis p.3). Impact Analysis answers the follow-up question: concept location said where the change *starts*; IA estimates what *else* it touches before any code is edited (ImpactAnalysis p.3–4).

2. Build the interaction picture around the seeds — with both lenses. The graph IA walks is the **class interaction graph** G = (X, I): classes as nodes, interactions as edges, where two classes interact if they have something in common (ImpactAnalysis p.5–6). The lab demands both analysis lenses because the two kinds of interaction need different instruments (AnalysisLab p.1):

- **Static analysis** — read the code without running it: imports, method calls, implemented interfaces, inheritance. This finds **dependency** interactions — "one depends on the other; there is a contract between them" (ImpactAnalysis p.5). (beyond slides — practical knowledge) The IDE's *Find Usages / Call Hierarchy* on each seed class enumerates its neighbors mechanically.
- **Dynamic analysis** — rerun the feature under the debugger (the previous lab's setup) and watch the data flow. This catches **coordination** interactions, where two classes share dataflow through a third without referencing each other — the deck's example is `b.paint(a.get());` inside a class C, creating an A–B edge that no import statement shows (ImpactAnalysis p.5, p.8).

Use interactions, not one-way dependency arrows: interactions propagate change **in both directions** — changing a method's signature ripples *up* to its callers, not only down to its callees — which is why IA is run on the undirected interaction diagram rather than the directed dependency diagram (ImpactAnalysis p.5, p.9).

3. Run the Figure 7.9 marking algorithm. The procedure, exactly as the deck's activity diagrams draw it (ImpactAnalysis p.12, p.16; AnalysisLab p.1):

```
1. Create the interaction diagram; mark ALL classes BLANK.
2. Mark the class(es) located during concept location CHANGED.   <- seed
3. Mark all BLANK neighbors of changed classes NEXT.             <- frontier
4. While any class is marked NEXT:
     pick one NEXT class and INSPECT it (open it, read it, judge it):
       - genuinely needs edits for this change      -> mark CHANGED,
            then mark its BLANK neighbors NEXT
       - needs no edits but relays the change       -> mark PROPAGATING,
            then mark its BLANK neighbors NEXT
       - not affected at all                        -> mark UNCHANGED
            (the diagrams record this as INSPECTED); frontier does not grow
5. No NEXT left -> stop.
   Estimated impact set = every class marked CHANGED or PROPAGATING.
```

Termination is guaranteed: each class moves monotonically BLANK → NEXT → {CHANGED, PROPAGATING, UNCHANGED/INSPECTED} at most once, and the graph is finite (ImpactAnalysis p.11–12). Note the labeling subtlety worth one exam mark: on the deck's diagrams the *verdict* branch is guarded [UNCHANGED] but the recorded *mark* is INSPECTED — verdict and bookkeeping word differ (ImpactAnalysis p.12, p.16).

4. Respect the mailman. The PROPAGATING mark is the algorithm's crucial refinement. Some classes are pure conduits — façades, delegators, panels that pass a changed object through unmodified. The deck's analogy: John writes to Paul, the mailman carries the letter, Paul's life changes — the change originated with John and propagated *through* the mailman, whose own life did not change (ImpactAnalysis p.10). Mark such a class UNCHANGED and the walk wrongly halts there, silently missing every impacted class on its far side; mark it PROPAGATING and its neighbors still enter the frontier (ImpactAnalysis p.10, p.12). When in doubt between UNCHANGED and PROPAGATING, ask: does changed *data or behavior travel through* this class to others?

5. Trace your walk concretely. A worked shape of the loop on a JHotDraw-flavored example (illustrative — substitute your feature's classes; the deck's own trace uses the store domain, ImpactAnalysis p.7, p.13):

```
Seed (from concept location):  ImageOutputFormat = CHANGED
Frontier: its neighbors -> NEXT: Drawing, SVGOutputFormat, ExportAction

inspect ExportAction   -> calls the format writer with unchanged arguments,
                          but relays the user's export trigger  -> PROPAGATING
                          its neighbor SVGDrawingPanel -> NEXT
inspect Drawing        -> read-only source of figures, interface untouched -> UNCHANGED
inspect SVGOutputFormat-> must apply the same change for SVG exports -> CHANGED
                          its BLANK neighbors -> NEXT ...
inspect SVGDrawingPanel-> no edits, nothing propagates further -> UNCHANGED
...
No NEXT left. Estimated impact set = {ImageOutputFormat, SVGOutputFormat, ExportAction}
```

(beyond slides — practical knowledge) **JRipples** is the course-literature tool for exactly this bookkeeping — it supports program comprehension during incremental change by managing the impact-set marks while you supply the judgments ([BBPR05] in the course literature list). Using it, or doing the marks by hand on a class diagram printout, is equivalent for the lab; what matters is that the marks exist and are auditable.

6. Weigh alternative change locations before committing. IA's second job is decision-making: when the change could be implemented in more than one place, run the estimate for each candidate and compare on

the deck's two criteria — **required effort of the change** and **clarity of the resulting code** — which often contradict each other; the lecture's example is Fahrenheit→Celsius, implementable cheaply at the display or cleanly where sensor data is converted, a short-term versus long-term conflict (ImpactAnalysis p.14–15). For a JHotDraw export-style change the analogous fork is: patch each format writer separately (low effort now, duplicated logic forever) or introduce one shared point all formats pass through (more effort now, one place to maintain). Record which you chose and *why* — this paragraph is report gold.

7. Fill in Table 1 by package. Aggregate your marks: for each package you entered, count the classes you visited (inspected — not just the changed ones) and write the comment the handout demands: what you learned about the package and how it contributes to your feature (AnalysisLab p.1–2). Illustrative shape:

Package name	# of classes	Comments
org.jhotdraw.samples.svg.gui	3	Hosts the export UI entry; triggers but does not implement export — propagating territory
org.jhotdraw.draw.io	4	The OutputFormat implementations; the real change site, one class per format
org.jhotdraw.draw	2	Drawing/figure model; supplies data, unaffected by the change itself

8. Close the loop into your plan. The estimated impact set is the to-do list Prefactoring and Actualization will edit and the basis of the effort estimate (ImpactAnalysis p.3–4). (beyond slides — practical knowledge) Write the set's size into your portfolio next to the change request — after Actualization you can compare estimate against the actual diff, and that estimate-versus-actual sentence is one of the most convincing reflective lines the exam report can contain.

Why & how — report-ready justifications

- I ran the **Figure 7.9 marking algorithm instead of browsing dependencies ad hoc**, because the marks (BLANK/NEXT/CHANGED/PROPAGATING/UNCHANGED) make the walk systematic, finite, and auditable — the same inputs reproduce the same estimated set — whereas unstructured browsing re-inspects some classes, skips others, and produces an estimate nobody can check (AnalysisLab p.1; ImpactAnalysis p.11–12).
- I walked the **interaction graph rather than the dependency graph**, because change propagates in both directions along an interaction — callers of a changed signature are impacted "upstream" — and because coordination edges like a dataflow between two classes inside a third never appear as dependencies at all; a dependency-only walk would have undercounted my impact set (ImpactAnalysis p.5, p.8–9).
- I used the **PROPAGATING mark for relay classes instead of marking them UNCHANGED**, because a conduit that needs no edits can still carry the change to classes beyond it — marking it UNCHANGED halts the search and silently hides genuinely impacted classes, which is precisely the under-estimation that causes regression surprises later (ImpactAnalysis p.10, p.12).
- I applied **both static and dynamic analysis rather than either alone**, because static reading finds contract dependencies but misses runtime coordination, while the debugger shows real dataflow but only on paths I trigger; combining them is the lab's own objective and gave the estimate both coverage and evidence (AnalysisLab p.1; ImpactAnalysis p.5, p.8).
- I compared **alternative change locations on required effort and resulting code clarity before editing**, choosing [the cleaner location / the cheaper location] because [reason], rather than defaulting to the first place that would work — making the short-term/long-term trade-off explicit instead of accidental (ImpactAnalysis p.14–15).

- I recorded the walk **by package with a learning comment each**, because the handout's Table 1 is not just a count — explaining how each package contributes to the feature is the comprehension evidence that distinguishes an analysis from a directory listing (AnalysisLab p.1–2).
- I treated the output as an **estimate to be verified, not a fact**, because the set is a prediction made before editing — verification runs across every phase of the change, and comparing the estimate to the actual diff afterwards told me [how accurate I was] (ImpactAnalysis p.3–4).

Copy-paste reflection for the report

In the impact-analysis lab I estimated the blast radius of my change before touching any code. My input was the initial impact set from concept location — [Class A] and [Class B] — and my task was to grow it into the estimated impact set by following the marking algorithm of Figure 7.9 [Raj13] (AnalysisLab p.1). I built the interaction picture around the seeds with both lenses the lab requires: statically, by reading imports, calls and interfaces ([tool/IDE feature] enumerated each class's neighbors), and dynamically, by re-running [feature] under the debugger to catch coordination edges — classes connected by dataflow through a third class, which no import statement reveals (ImpactAnalysis p.5, p.8). Then I ran the loop: seeds CHANGED, neighbors NEXT, each inspected class judged CHANGED, PROPAGATING or UNCHANGED. The judgment that mattered most was [conduit class]: it needs no edits itself, but it relays [data/trigger] onward, so I marked it PROPAGATING rather than UNCHANGED — stopping there would have hidden [downstream class] from the estimate entirely (ImpactAnalysis p.10). I also weighed two places to implement the change — [option 1] versus [option 2] — and chose [choice] because [clarity/effort reason], accepting the trade-off between immediate effort and the clarity of the resulting code (ImpactAnalysis p.14–15). My result, recorded in the lab's Table 1, spans [N] packages and [M] visited classes: [package — what I learned]. After actualization I compared the estimate to the real diff: [accurate / one class over / one class under], which taught me [lesson].

Likely exam questions about this lab — with answers

Q: What is impact analysis, and where does it sit in the change process? Impact analysis is "identifying the potential consequences of a change, or estimating what needs to be modified to accomplish a change" (AnalysisLab p.1). It is the phase after Concept Location and before Prefactoring: the located classes are its input (the initial impact set), it analyzes class interactions, and it produces the estimated impact set that the implementation phases will actually edit (ImpactAnalysis p.3). Verification runs alongside it, as alongside every phase (ImpactAnalysis p.3).

Q: Distinguish the initial impact set from the estimated impact set. The initial impact set is what concept location hands over — the seed classes where the change visibly starts, often a single class. The estimated impact set is the output of impact analysis: the initial set grown outward along interactions until the frontier stops, covering everything the change is predicted to touch (ImpactAnalysis p.4). The initial set is always a subset of the estimated set, and the difference between them is exactly the ripple effect IA exists to predict.

Q: Describe the marking algorithm you followed. All classes start BLANK; the concept-location classes are marked CHANGED; their BLANK neighbors become NEXT. While any NEXT exists, one is selected and inspected: if it needs edits it becomes CHANGED and its BLANK neighbors become NEXT; if it relays the change without needing edits it becomes PROPAGATING and likewise expands the frontier; if unaffected it is recorded as UNCHANGED/INSPECTED and the branch closes. When no NEXT remains, the estimated impact set is every class marked CHANGED or PROPAGATING (ImpactAnalysis p.12, p.16; AnalysisLab p.1). It terminates because marks only move forward and the graph is finite.

Q: What is a propagating class and why does the algorithm need the mark? A propagating class relays a change between its neighbors without needing new code itself — the deck's mailman, who carries John's letter to Paul while his own life stays unchanged (ImpactAnalysis p.10). Without the mark, such conduits would be judged UNCHANGED, the walk would halt, and impacted classes beyond them would be missed — an under-estimated set and a future regression. PROPAGATING records "no edit here, but keep searching past it" (ImpactAnalysis p.12).

Q: Why is IA run on interactions rather than dependencies? Two reasons. Interactions propagate change in both directions — changing a class's interface impacts its callers, so impact flows backwards up a dependency arrow — and some interactions are coordinations with no dependency at all, like two classes joined by dataflow inside a third (`b.paint(a.get());`), an edge a dependency diagram never draws (ImpactAnalysis p.5, p.8–9). A one-way, dependency-only walk would systematically undercount the estimated set.

Q: The lab asks for static *and* dynamic analysis — why both? Static analysis reads source without executing it and reliably finds contract dependencies, but it can miss coordination interactions whose only evidence is runtime dataflow; dynamic analysis observes the running program and surfaces exactly those edges, but only on paths you trigger (AnalysisLab p.1; ImpactAnalysis p.5, p.8). Each lens covers the other's blind spot, so the combined estimate is less likely to be undercounted.

Q: How does impact analysis support choosing between alternative implementations? When a change can be made in more than one location, IA estimates each candidate and compares on two criteria: the required effort of the change and the clarity of the resulting code — which often conflict, as in the Fahrenheit→Celsius example where patching the display is easier but centralizing temperature conversion is clearer (ImpactAnalysis p.14–15). IA's job is to surface both options with costs so the trade-off between short-term and long-term goals is decided deliberately, not by default.

Q: Why is the output called an *estimated* impact set? Because it is a prediction made before the code is edited — it can be too large (over-cautious marks) or too small (a missed ripple). Verification, which spans every phase of the change process, is what catches mis-estimates later, for example as regression faults found at baseline (ImpactAnalysis p.3–4). Comparing the estimate against the actual diff after actualization is the honest way to close the loop.

Pitfalls, mistakes, and what the examiner looks for

- **Stopping at the initial set.** Editing only the concept-location classes and declaring victory skips the entire phase; the surprise then arrives as regressions in classes never inspected. The whole point is that the initial set is a *subset* of what the change touches (ImpactAnalysis p.4).
- **Walking dependencies one-way.** Following only outgoing arrows misses the callers a signature change breaks. Interactions are undirected for IA precisely because impact flows both ways (ImpactAnalysis p.5, p.9).
- **Killing the walk at a conduit.** Marking a relay class UNCHANGED instead of PROPAGATING is the single most damaging judgment error — it silently prunes every impacted class behind the conduit (ImpactAnalysis p.10).
- **Marking everything CHANGED to be safe.** Over-caution inflates the estimate until it stops being a plan; an estimated set covering half the codebase predicts nothing. The marks have meaning only if UNCHANGED is used honestly.
- **A Table 1 without comments.** The handout explicitly asks what you *learned* about each package and how it contributes to the feature (AnalysisLab p.1–2). Counts without comments read as a directory listing, not an analysis.

- **Confusing the UNCHANGED verdict with the INSPECTED mark.** On the algorithm diagrams the branch guard says [UNCHANGED] but the recorded mark is INSPECTED; the interactive variant uses INSPECTED for both (ImpactAnalysis p.12, p.16). Listing the marks completely — BLANK, NEXT, CHANGED, UNCHANGED, plus PROPAGATING and INSPECTED — is the safe exam answer.
- **What distinguishes a good report answer:** naming the algorithm and its marks, narrating one real PROPAGATING judgment, quoting your Table 1, stating the alternatives you weighed on effort versus clarity, and closing with estimate-versus-actual. That sequence demonstrates the phase as engineering, not ritual (What-To-Expect p.1, p.4).

Theory links

- **The change mini-cycle.** IA is the third phase — after Initiation and Concept Location, before Prefactoring and Actualization — with Verification cross-cutting all of them down the right-hand side of the process diagram (ImpactAnalysis p.3).
- **Graph formalism.** The class interaction graph G = (X, I) and the neighborhood N(A) = {B | (A,B) ∈ I} are what make IA algorithmic: the frontier the algorithm grows is exactly the neighborhood of each newly marked class (ImpactAnalysis p.6–7).
- **Figure 7.9 [Raj13].** The lab's named procedure is Rajlich's marking algorithm; the deck presents it twice — the propagating-classes version and the interactive Computer/Programmer swimlane version, which shows IA as human judgment supported by mechanical bookkeeping (AnalysisLab p.1; ImpactAnalysis p.12, p.16).
- **Interactions: dependency and coordination.** "Something in common" comes in two flavors — contracts and coordination — and the coordination flavor is why dynamic analysis is in the lab's objective (ImpactAnalysis p.5, p.8; AnalysisLab p.1).
- **Short-term vs. long-term goals.** The effort-versus-clarity criteria connect IA to the course's larger themes: prefactoring, refactoring, and technical debt all manage the same conflict the Fahrenheit→Celsius example exposes (ImpactAnalysis p.14–15).
- **JRipples.** The course literature's tool for impact-set marking during incremental change ([BBPR05]); using it in the lab is the tool-supported version of the same algorithm.
- **Handoff forward.** The estimated impact set is the work list for Prefactoring/Actualization and the basis of effort estimates in planning — IA is where a change first acquires a budget (ImpactAnalysis p.3–4).

CI Lab (Lecture 3) — a GitHub Actions pipeline for the Maven build

The assignment — what the handout asks

The CILab ("Impact Continues Integration Lab") is a one-page handout. Its introduction is a quotable mini-history: "continuous integration (CI) is the practice of merging all developers' working copies to a shared mainline several times a day [Tho]. Grady Booch first proposed the term CI in his 1991 method although he did not advocate integrating several times a day [Boo]. Extreme programming (XP) adopted the concept of CI and did advocate integrating more than once per day — perhaps as many as tens of times per day [Bec99]" (CILab p.1).

Two objectives (CILab p.1):

1. **Understand what CI is.**
2. **Setup a simple CI pipeline.**

The classwork is five numbered steps (CILab p.1):

1. Go to **[Building and testing Java with Maven]** — GitHub's documented guide for Actions + Maven.
2. **Add a `*.yml` file** to your repository path `<YOUR_PROJECT>/.github/workflows/` "to tell GitHub Actions CI what to do."
3. **Configure the `*.yml` to automatically build your project for each pull request** (use Maven).
4. To use shared jars from GitHub Packages, **create a `.maven-settings.xml` file in the project root folder** — see [Working with the Apache Maven registry].
5. **Configure the `*.yml` to execute tests automatically.**

The deliverable is a working pipeline in the graded repository: a version-controlled workflow file that builds every pull request and runs the tests. The exam relevance is unusually direct — "how did you create a pipeline" was the final question in an example report the lecturer showed, and continuous delivery and pipelines were singled out as important topics (What-To-Expect p.4, p.6).

Step-by-step walkthrough — how to do it

1. Work on a branch, like any other change. (beyond slides — practical knowledge) The pipeline is itself a change to the repository, so it follows the same GitHub flow as code: `git checkout -b ci/maven-pipeline`, commit the workflow there, and open a pull request — which, satisfyingly, becomes the first PR the new pipeline builds.

2. Create the workflow file in the path GitHub scans. The location is load-bearing: GitHub Actions only discovers workflows under `.github/workflows/` at the repository root (CILab p.1).

```
mkdir -p .github/workflows
# create the file (any *.yml name works; "maven.yml" is conventional)
```

3. Write the workflow — build on every pull request, tests on. Adapted from GitHub's "Building and testing Java with Maven" guide, the document the handout's step 1 sends you to (CILab p.1); pinned to the course toolchain of JDK 11 + Maven from the IntroLab (IntroLab p.1):

```
name: Java CI with Maven

on:
  pull_request:            # handout step 3: build each pull request
  push:
    branches: [ "main" ]   # also guard the mainline itself

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up JDK 11
        uses: actions/setup-java@v4
        with:
          java-version: '11'
          distribution: 'temurin'
          cache: maven       # keep the build fast: cache ~/.m2

      - name: Build and test with Maven
        run: mvn --batch-mode clean verify -s .maven-settings.xml
```

Line rationale (beyond slides — practical knowledge): `on: pull_request` is handout step 3 verbatim — every proposed integration is built *before* it can merge; `actions/checkout` gives the runner the repository's code,

because the build server "only gets code from the repo" (ContinuousIntegration p.6); `setup-java` pins the same JDK 11 the IntroLab pinned locally, so the server build is the local build; `cache: maven` keeps feedback fast by reusing downloaded dependencies; and `mvn clean verify` — *not* `-DskipTests` — runs the lifecycle through the test phase, which is handout step 5 and CI's self-testing principle in one flag's absence (CILab p.1; ContinuousIntegration p.9).

4. Add `.maven-settings.xml` for GitHub Packages. Handout step 4: shared jars from GitHub Packages require Maven to know the registry and credentials (CILab p.1). Following [Working with the Apache Maven registry], create `.maven-settings.xml` in the project root:

```
<settings xmlns="http://maven.apache.org/SETTINGS/1.0.0">
  <servers>
    <server>
      <id>github</id>
      <username>${env.GITHUB_ACTOR}</username>
      <password>${env.GITHUB_TOKEN}</password>
    </server>
  </servers>
  <profiles>
    <profile>
      <id>github</id>
      <repositories>
        <repository>
          <id>github</id>
          <url>https://maven.pkg.github.com/OWNER/REPOSITORY</url>
        </repository>
      </repositories>
    </profile>
  </profiles>
  <activeProfiles>
    <activeProfile>github</activeProfile>
  </activeProfiles>
</settings>
```

(beyond slides — practical knowledge) Two details matter: credentials come from **environment variables**, never literals — in Actions, expose the automatic token to the build step with `env: GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}` — and the build must be told to *use* this file, hence `-s .maven-settings.xml` on the `mvn` line, because a file in the project root is not Maven's default settings location (`~/.m2/settings.xml`).

5. Push, open the PR, and watch the first run. Push the branch, open the pull request, and the Checks tab shows the workflow executing: checkout, JDK setup, then the Maven build streaming the same output you saw locally in the IntroLab. The cycle the lecture draws — **Commit → Build → Test → Report → back to Development**, all revolving around source control (ContinuousIntegration p.2) — is now literally visible in the PR.

6. Prove the net catches. (beyond slides — practical knowledge) A pipeline demonstrated only green proves nothing. Push a commit with a deliberately failing test, watch the run go red and the PR get blocked-by-failure, then revert and watch it go green. Screenshot or link both runs in your portfolio: it is one of the strongest "how do you know it works" answers available for the report.

7. Make green mandatory. (beyond slides — practical knowledge) In the repository settings, add a branch protection rule for `main` requiring the workflow's status check to pass before merging. This upgrades the pipeline from advisory to enforced: the build server becomes the neutral arbiter that "settles disputes" about whether code works — not anyone's laptop (ContinuousIntegration p.6).

8. Keep it fast, deliberately. The fifth CI principle exists because slow builds quietly kill principle three — developers stop building every commit when feedback takes too long (ContinuousIntegration p.3). The lab-scale levers: the Maven dependency cache (step 3), `--batch-mode` to skip interactive output, and resisting the temptation to bolt heavy static-analysis jobs onto every PR — the deck's own hardware slide warns static analysis demands "lots and lots" of CPU and pleads "Please, KEEP IT FAST" (ContinuousIntegration p.11).

Why & how — report-ready justifications

- I built the pipeline with **GitHub Actions rather than a separate build server such as Jenkins**, because the workflow file lives version-controlled in the same repository as the code it builds — the pipeline's history is reviewable like any other change — and because Actions is natively wired to the pull-request events our GitHub flow already produces (CILab p.1; Git Lab p.16).
- I triggered the build **on every pull request rather than only on pushes to main**, because the point of CI is to verify each integration *before* it lands — building after merge discovers breakage when it is already shared, which is integration hell on a delay (CILab p.1; ContinuousIntegration p.2).
- I made the build **self-testing by running `mvn verify` instead of skipping tests**, because a green build must mean working software, not merely compiling software — and unit tests are the right verification layer for CI because they are fast and pinpoint the broken unit (CILab p.1; ContinuousIntegration p.9).
- I resolved shared jars through a **`.maven-settings.xml` against GitHub Packages rather than committing jars or relying on someone's laptop**, because the build server must be able to assemble everything from the repository and the registry alone — the source-of-record principle — and because binary jars in Git are exactly what the version-control lab forbids (CILab p.1; ContinuousIntegration p.6; Git Lab p.6).
- I **pinned JDK 11 in the workflow to match the local toolchain**, because CI's authority depends on the server building what developers build — a version-drifted server produces failures (or worse, passes) that mean nothing (IntroLab p.1; ContinuousIntegration p.6).
- I **cached the Maven repository and kept the job lean instead of adding every analysis tool**, because feedback speed is a CI principle in its own right: a slow build defeats build-every-commit by making developers stop waiting for it (ContinuousIntegration p.3, p.11).
- I **verified the pipeline with a deliberate red run before trusting it**, because an untested safety net is a hypothesis: pushing a failing test and watching the PR block, then reverting to green, demonstrated the net actually catches (beyond slides — practical knowledge).

Copy-paste reflection for the report

In the continuous-integration lab I automated the verification of every change entering our repository. Following the handout's steps, I started from GitHub's "Building and testing Java with Maven" guide and added a workflow file, [maven.yml], under .github/workflows/ in our JHotDraw fork — version-controlled alongside the code it builds (CILab p.1). I configured it to trigger on every pull request, so each proposed integration is built and tested before it can merge: the runner checks out the repository, sets up JDK 11 to match our local toolchain from the introduction lab, restores the Maven dependency cache, and runs mvn --batch-mode clean verify. I deliberately did not skip tests — a green run must mean working software, not merely compiling software, which is the self-testing-build principle (ContinuousIntegration p.9). Because the build uses shared jars from GitHub Packages, I added a .maven-settings.xml in the project root pointing Maven at the registry, with credentials supplied through the workflow's environment rather than committed to the repository (CILab p.1). I proved the pipeline works in both directions: [describe your red run — e.g. a deliberately failing test] turned the pull request red and blocked the merge, and reverting it restored green. [Optionally: I then made the check mandatory through branch protection on main.] The practical effect

showed up when [incident — e.g. a teammate's PR broke a module they had not touched]: the pipeline reported it within minutes of the push, while the cause was still fresh, instead of at the end of the iteration (ContinuousIntegration p.7).

Likely exam questions about this lab — with answers

Q: How did you create your pipeline? This is a known template question — it was the final question in an example report the lecturer showed (What-To-Expect p.6). Answer with the five lab steps made concrete: I followed GitHub's "Building and testing Java with Maven" guide; added a workflow YAML under `.github/workflows/`; configured it to build the project with Maven on every pull request; added a `.maven-settings.xml` in the project root so the build resolves shared jars from GitHub Packages; and configured the workflow to execute the test suite automatically (CILab p.1). Then state the verification: a deliberate red run, then green, then branch protection.

Q: What is continuous integration, and where does the term come from? CI is the practice of merging all developers' working copies to a shared mainline several times a day, with each integration verified by an automated build (CILab p.1; ContinuousIntegration p.2). Grady Booch first proposed the term in his 1991 method, though without advocating multiple daily integrations; Extreme Programming adopted the concept and pushed the frequency to many times per day (CILab p.1). Its value is killing integration hell: frequent small merges keep conflicts and breakages small and freshly diagnosable.

Q: What are the five principles of CI? Environments based on stability — promote code through progressively stricter environments; maintain a code repository — one shared source of record; commit frequently and build every commit — small integrations, caught immediately; make the build self-testing — green means working, not just compiling; keep the build fast — slow feedback defeats frequent building (ContinuousIntegration p.3). They interlock: frequent commits are only safe if the build is self-testing and fast, which requires the single repository and the promotion pipeline.

Q: Why build on every pull request rather than nightly or before release? The deck's justification is the agile principle "if it hurts, do it more often": integration is painful, so doing it constantly keeps each instance small, and frequent builds shrink the time between defect introduction and detection so the cause is still fresh when the alarm rings (ContinuousIntegration p.7). Mechanically, the PR trigger verifies the integration *before* it lands on the mainline, which is the only point where a failure is still private and cheap.

Q: Why are unit tests the right tests for the CI build? The deck distinguishes system tests — end-to-end, thorough, but taking minutes to hours — from unit tests, which are fast (no database or filesystem) and focused, pinpointing exactly which unit broke (ContinuousIntegration p.9). Speed serves the keep-it-fast principle; focus makes a red build instantly diagnosable. The deeper rationale is defect-detection arithmetic: individual programmers find fewer than half of their own bugs, and combining three or more quality methods pushes defect removal above 90% — so the build's automated tests layer on top of human inspection rather than replacing it (ContinuousIntegration p.8).

Q: What is the `.maven-settings.xml` for? It tells Maven about the GitHub Packages registry and the credentials to use, so the build — locally and on the CI runner — can resolve shared jars from the registry instead of from any individual machine (CILab p.1). It lives in the project root and is passed to Maven explicitly with `-s .maven-settings.xml`; credentials are injected from environment variables (the Actions `GITHUB_TOKEN`), never committed (beyond slides — practical knowledge). It is the source-of-record principle applied to dependencies: everything the build needs is fetchable from the repository plus the registry (ContinuousIntegration p.6).

Q: How does the pipeline contribute to the repository the examiner grades? Pipeline setup is explicitly on the list of what the repository is evaluated for, alongside code changes, refactoring, and testing (What-To-Expect p.5). A merged workflow file, a history of green check runs on PRs, and a demonstrable red run on a failure show maintenance discipline made automatic: every later change in the repository — refactorings, tests, the portfolio feature — carries the pipeline's verification stamp in its PR.

Pitfalls, mistakes, and what the examiner looks for

- **Workflow in the wrong place.** Actions only reads `.github/workflows/` at the repository root (CILab p.1). A YAML anywhere else simply never runs — and a pipeline that never ran is visible in the graded repository's empty Actions tab.
- **Building only pushes to main.** Handout step 3 says *each pull request* (CILab p.1). Triggering only on push verifies integrations after they have already landed, which forfeits the entire point of gating.
- **`-DskipTests` in the pipeline.** Carrying the IntroLab's bootstrap flag into CI produces a build that is not self-testing — green stops meaning anything (ContinuousIntegration p.9). The skip flag was a day-one convenience; in the pipeline it is a contradiction, and handout step 5 exists precisely to remove it (CILab p.1).
- **Credentials in the settings file.** A personal access token committed inside `.maven-settings.xml` is a security failure sitting in a graded repository. Use environment-injected secrets; the file should contain `${env.…}` references, never values (beyond slides — practical knowledge).
- **A green badge nobody tested.** If the pipeline has never been seen red, you do not know it can fail. The deliberate failing-test run is cheap and is the difference between "I configured CI" and "I verified my CI catches defects."
- **A slow, bloated job.** Piling static analysis and long system tests into the PR build violates keep-it-fast and trains the team to ignore the checks (ContinuousIntegration p.3, p.11). Heavy analysis belongs in separate or scheduled jobs.
- **What distinguishes a good report answer:** reciting your own YAML's decisions (trigger, JDK pin, cache, verify), the settings-file rationale, and a verification story with a red run — in the why/how shape: each configuration line is a decision with a rejected alternative and a benefit (What-To-Expect p.1, p.4).

Theory links

- **The CI cycle.** Commit → Build → Test → Report → back to Development, revolving around source control — the deck's picture is exactly what the Actions run renders per pull request (ContinuousIntegration p.2).
- **The five principles.** The lab realizes four directly: the repository (Git Lab), build-every-commit (PR trigger), self-testing build (step 5), keep it fast (cache, lean job); environment promotion is previewed as the Dev → Test → Stage → Prod gradient (ContinuousIntegration p.3–4).
- **The build server as arbiter.** "The code builds on my box" is settled by a neutral server that only gets code from the repo — the lab's runner plus branch protection implements it (ContinuousIntegration p.6).
- **"If it hurts, do it more often."** The frequency justification: frequent integration shrinks the defect-introduction-to-detection gap, which is why automation must make each build free (ContinuousIntegration p.7).
- **Testing economics.** Unit tests verify builds because they are fast and focused; layered quality methods (inspections plus testing) push defect removal above 90% where any single method — especially self-checking — stays under 50% (ContinuousIntegration p.8–9).
- **Tool landscape.** The deck's named tooling — JUnit for unit tests; Checkstyle, FindBugs, PMD for static analysis; Cobertura for coverage; SONAR for hotspots — is the menu the pipeline can grow into after the

lab's minimal build (ContinuousIntegration p.10, p.12–14).

- **Roots in the Git and Intro labs.** The pipeline presupposes the repository discipline of the Git Lab (clean history, PRs, no binaries) and the reproducible Maven/JDK toolchain of the IntroLab; it is those labs' manual verification loop, made automatic and mandatory (Git Lab p.19; IntroLab p.1).